



VIRTUAL-LAB

Collaborative 2D Physics Sandbox for Physics Education

Project Report

Team Members:

Eshika Katekhaye

Aadrika Singh

Mona Jat

Contents

1	Introduction	2
2	Problem Statement and Motivation	2
3	System Overview	3
3.1	Feature Summary	3
3.2	Preset Experiments	4
3.3	User Flow	4
4	System Architecture	5
4.1	High-Level Architecture	5
4.2	Frontend Architecture	6
4.3	Backend Architecture	6
4.4	Database Design	7
5	Key Technical Implementations	7
5.1	Real-Time Canvas Synchronisation	7
5.2	Authentication and Role-Based Access Control	8
5.3	Physics Simulation and Analytics	9
5.4	Chaos Events	9
6	Screenshots and Interface	10
6.1	Login and Registration	10
6.2	Instructor Dashboard	10
6.3	Physics Canvas	11
6.4	Analytics Panel	12
7	DevOps and Deployment	13
7.1	Docker Containerisation	13
7.2	CI/CD Pipeline	14
8	Testing	14
8.1	Unit Tests - OT Engine	14
8.2	Integration Tests - Auth Routes	14
9	Limitations and Future Work	15
10	Conclusion	16

Appendix: Technology Stack Summary

17

1. Introduction

Teaching physics in an online or hybrid setting presents a persistent challenge that goes beyond poor internet connections or student engagement. The deeper problem is the absence of physical intuition. When a student reads that kinetic energy equals $\frac{1}{2}mv^2$, they understand the formula. What they do not understand - at least not without experience - is how that energy transforms into potential energy as a pendulum rises, or how it dissipates during an inelastic collision. That kind of understanding comes from watching, touching, and manipulating physical systems directly.

Traditional online tools address this in limited ways. Simulation libraries like PhET Interactive Simulations provide pre-built experiments, but they are designed for individual use and do not support live collaboration. An instructor cannot guide thirty students through the same simulation simultaneously, adjust parameters in real time, or watch a student's work evolve and respond to it in the moment.

VIRTUAL-LAB was built to fill this gap. It is a real-time, multi-user 2D physics sandbox running entirely in the browser. Instructors and students share a live canvas where they can place physical bodies, define mechanical constraints, and run simulations together. When an instructor loads a pendulum preset, it appears on every connected student's screen within milliseconds. A live energy conservation graph updates as the simulation runs. The instructor can lock the canvas for a focused demonstration, trigger dramatic physics events that every student experiences simultaneously, and save experiments for students to revisit later.

This report documents the design decisions, system architecture, implementation details, and key technical challenges encountered while building VIRTUAL-LAB.

2. Problem Statement and Motivation

The project was motivated by three specific limitations observed in current online physics education tools:

- 1. Static content cannot demonstrate dynamic systems.** A video of a pendulum shows a pendulum. It does not let a student change the length, mass, or gravitational constant and immediately see the effect on the period. Genuine learning requires genuine interactivity.

2. **Existing simulation tools are not designed for classrooms.** Tools like PhET are excellent for individual exploration but have no concept of a shared live session. An instructor cannot say “watch what happens when I increase the spring stiffness” and have every student observe the same canvas change in real time.
3. **There is no feedback loop between student experiments and instructor evaluation.** Students run simulations locally and submit screenshots at best. Instructors have no way to see the actual state of a student’s experiment, or check whether the student observed what the assignment required them to observe.

VIRTUAL-LAB addresses all three. It provides an interactive canvas accessible to the entire class simultaneously, a role-based permission model that gives instructors control during demonstrations, and an experiment library where instructors can publish templates and students can save and share their work.

3. System Overview

3.1 Feature Summary

The system supports two user roles - **students** and **instructors** - each with a distinct set of capabilities.

Table 1: Feature matrix by user role

Feature	Student	Instructor
Join a room via 6-character code	✓	✓
Add physics bodies to canvas	✓	✓
Drag and move bodies in real time	✓	✓
View live velocity and acceleration vectors	✓	✓
View energy conservation graph	✓	✓
Adjust physics parameters (gravity, stiffness, density)	✓	✓
Save experiments to personal library	✓	✓
Browse and clone experiment library	✓	✓
Create and name rooms		✓
Lock canvas for demonstration mode		✓
Trigger chaos events		✓
Publish experiment templates		✓
Grade student submissions		✓

*Some elements like constraints are not being restored. Full experiment state restoration on reload is planned as future work.

3.2 Preset Experiments

Five preset scenarios are available with a single click, each loading a configured physics scene on all connected canvases simultaneously:

- **Free Fall** - multiple body types (box, circle, triangle) dropped under gravity to demonstrate mass independence of free fall
- **Simple Harmonic Motion (SHM)** - a mass attached to an anchored spring, oscillating to demonstrate Hooke's Law
- **Coupled Springs** - two masses connected by a spring, showing energy transfer between coupled oscillators
- **Pendulum** - a rigid-rod pendulum with live energy graph demonstrating conservation of energy
- **Collision** - multiple bodies demonstrating elastic and inelastic collision dynamics

3.3 User Flow

A typical classroom session using VIRTUAL-LAB proceeds as follows:

- Step 1.** The instructor logs in, creates a room named for the class (e.g. "Mechanics Lab 3A"), and shares the generated 6-character join code with students.
- Step 2.** Students register or log in, enter the code on their dashboard, and join the room. The canvas syncs immediately.
- Step 3.** The instructor loads a preset - for example, the Pendulum preset - and all students see the pendulum appear on their canvas simultaneously.
- Step 4.** The instructor adjusts physics parameters using sliders: gravity, spring stiffness, density, bounciness, and friction. Students observe the effect in real time.
- Step 5.** If the instructor wants a focused demonstration, they lock the room. Student edit controls disable immediately and a "Room locked" indicator appears on their screens.
- Step 6.** The instructor may trigger a Chaos Event - Flip Gravity, Shockwave, Freeze All, or Zero Gravity - to illustrate a concept dramatically. The event broadcasts to every user in the room simultaneously.
- Step 7.** Students save their experiment state to the library for later review or submission.

4. System Architecture

4.1 High-Level Architecture

The system is divided into three tiers: a React frontend, a Node.js backend, and a MongoDB database. Communication between the frontend and backend uses two channels simultaneously - a standard HTTP REST API for data operations, and a persistent WebSocket connection managed by Socket.io for real-time event delivery.

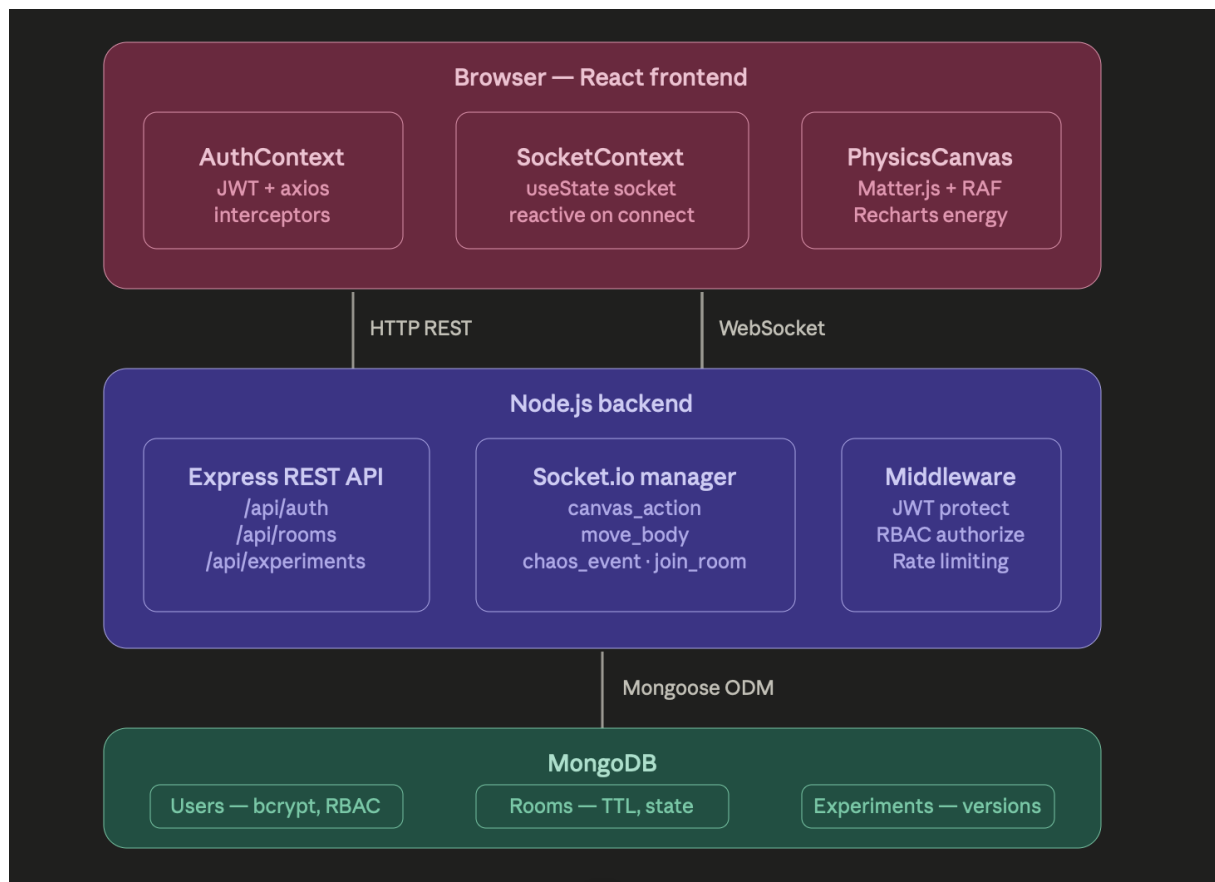


Figure 1: Three-tier system architecture. The browser communicates with the backend via HTTP for data operations and via WebSocket for real-time canvas events. All persistent state lives in MongoDB.

The key design insight is that the two communication channels serve different purposes. REST handles operations where persistence matters - creating a room, saving an experiment, registering a user. The WebSocket handles operations where latency matters - broadcasting a body position, propagating a chaos event, syncing a drag movement. Mixing these responsibilities into one channel would make both worse.

4.2 Frontend Architecture

The frontend is built in React 18 using the Context API for global state management. Two contexts sit at the root of the application tree:

AuthContext manages the user session. It stores the JWT token in `localStorage` and attaches it to every outgoing HTTP request via an axios interceptor. A global response interceptor catches 401 errors and redirects to the login page automatically, so individual components never need to handle session expiry.

SocketContext manages the single Socket.io connection for the session. A critical design decision here was storing the socket instance in `useState` rather than `useRef`. The Socket.io client connects asynchronously - when a component first mounts and calls `useSocket()`, the connection may not yet exist. A ref value passed through context is a snapshot at render time; it would be `null` and would never trigger a re-render when the connection became live. By storing the instance in React state, the component tree re-renders the moment the connection is established, ensuring all socket listeners are registered against a real, live socket.

The main physics component, **PhysicsCanvas**, integrates Matter.js directly on the browser's main thread. It establishes the physics engine, creates static boundaries, and registers event listeners for mouse interaction. A `requestAnimationFrame` loop redraws the canvas each frame with velocity vectors (green) and acceleration vectors (red) overlaid on each body in real time.

4.3 Backend Architecture

The backend is a Node.js/Express server with Socket.io running on the same HTTP server instance. The REST API exposes four route groups:

- `/api/auth` - registration, login, JWT token verification
- `/api/rooms` - room creation, joining by code, state persistence, lock/unlock
- `/api/experiments` - save, version, publish, clone, submit, and grade
- `/api/analytics` - fetch stored timeseries data for a room session

The Socket.io manager handles all real-time events. Rooms map directly to Socket.io rooms, so broadcasting to all users in a session requires a single `io.to(roomCode).emit()` call. Authentication reuses the same JWT on the socket handshake, verified by a middleware layer before the connection is accepted. Every socket event handler has access to a verified `socket.user` object without any additional authentication logic.

All route groups except `move_body` socket events write through a `protect` middleware

that verifies the JWT and checks the user still exists in the database. Role-specific routes use an additional `authorize(...roles)` middleware that returns 403 if the caller's role is not permitted.

4.4 Database Design

MongoDB was chosen for its flexible document schema, which suits the variable and nested structure of physics world state. Three collections store all persistent data:

Users stores hashed credentials and role information. Passwords are hashed with bcrypt at a cost factor of 12 before storage. The `passwordHash` field uses Mongoose's `select: false` option, ensuring it is never returned in queries unless explicitly requested - a deliberate security measure.

Rooms stores the current physics world state as an embedded JSON document, a version counter, a participant list, and an expiry timestamp. A MongoDB TTL index on the expiry field automatically deletes rooms after 24 hours without any cron job or cleanup task.

Experiments stores saved physics states with a full version history array. Each save appends a new version entry with a state snapshot, similar to a Git commit. Instructor-published templates carry a `submissions` subdocument array for student work and grades.

5. Key Technical Implementations

5.1 Real-Time Canvas Synchronisation

Ensuring that all users in a room see an identical canvas at all times was the most technically demanding aspect of the project. It required solving two distinct synchronisation problems with different solutions.

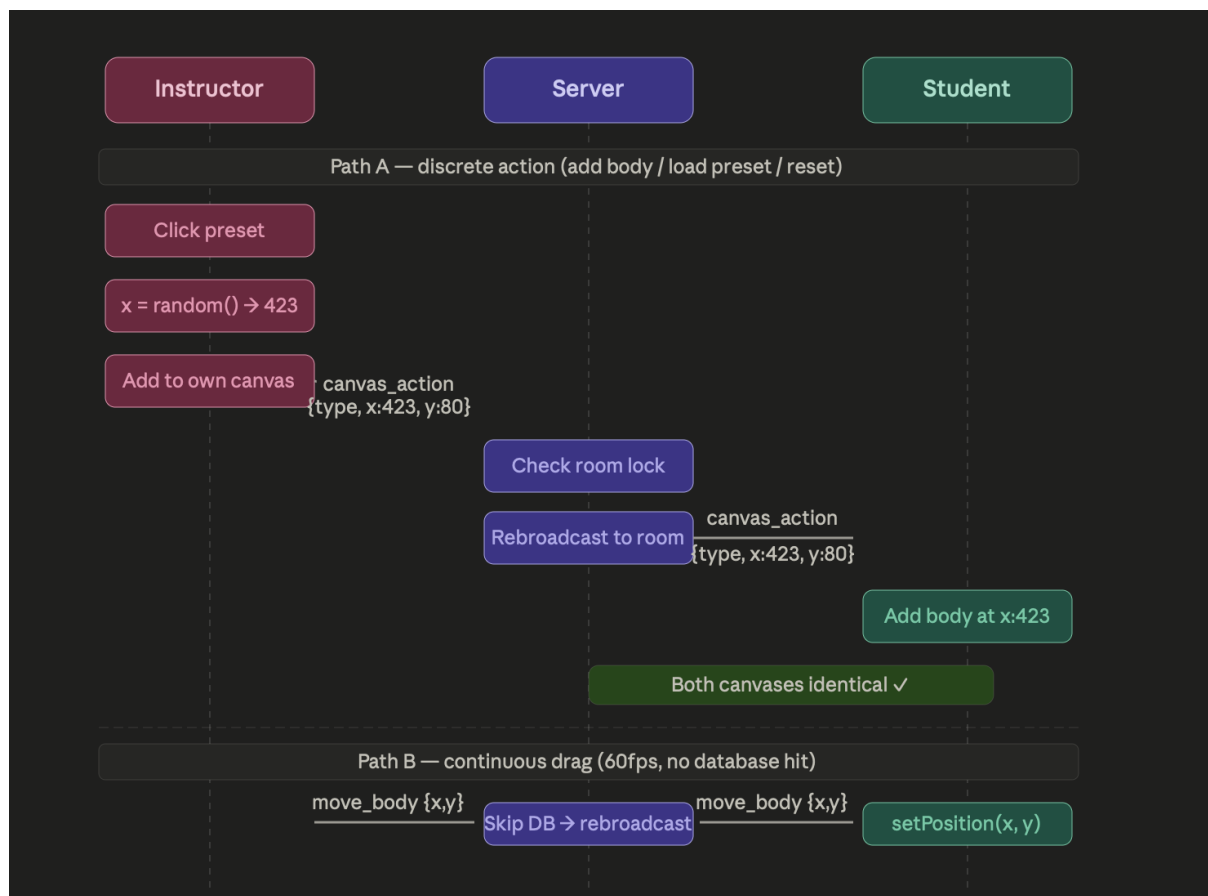


Figure 2: Real-time sync sequence. Path A handles discrete actions (add body, load preset, reset) by sharing explicit coordinates. Path B handles continuous drag events at 60fps by bypassing the database entirely.

5.2 Authentication and Role-Based Access Control

The authentication system uses JSON Web Tokens signed with a server-side secret. On login, the server signs a payload containing the user ID and role with a seven-day expiry. The client stores this token in `localStorage` and attaches it to every subsequent request via an axios interceptor.

Three roles exist in the system: `student`, `instructor`, and `admin`. Role assignment happens at registration and cannot be changed by the user. The `authorize(...roles)` middleware wraps any endpoint that requires a specific role, returning 403 Forbidden if the caller does not qualify.

On the WebSocket side, the same JWT is passed in the Socket.io handshake `auth` object and verified before the connection is accepted. This means socket event handlers receive a verified `socket.user` object without needing to re-authenticate per event.

5.3 Physics Simulation and Analytics

The physics engine is Matter.js, a mature 2D rigid body library. The simulation runs on the browser's main thread at 60 updates per second using Matter.js's own **Runner**.

Each body is assigned a label string as its unique identifier. This label is used across clients to match incoming position updates to the correct local body — the label is the coordination key that makes multi-user drag sync possible.

Vector visualisation is implemented in the Matter.js **afterRender** event. For each non-static body, velocity and acceleration vectors are drawn directly onto the canvas context every frame. Acceleration is derived by comparing consecutive velocity readings with exponential smoothing (80%/20% blend) to suppress jitter from the fixed-timestep integrator.

Energy analytics are computed in the same render loop. Kinetic energy is calculated as $KE = \frac{1}{2}mv^2$ using the body's mass and current speed. Potential energy is approximated as $PE = mgh$ where h is the body's height above the canvas floor, scaled to appropriate units. Both values are fed into a Recharts **LineChart** via React state updates, producing a live graph that updates every frame for the currently tracked body. The flat total energy line on the graph provides a visual demonstration of the law of conservation of energy.

Physics parameters are adjustable in real time via sliders: gravity (including negative values for inverted gravity), spring stiffness, body density, coefficient of restitution (bounciness), and friction. Changes apply immediately to the running simulation and are visible to all connected users.

5.4 Chaos Events

Chaos Events are instructor-only actions that broadcast dramatic physics effects to every user in the room simultaneously via a single `io.to(roomCode).emit()` call. Five events are implemented:

- **Flip Gravity** - inverts `engine.gravity.y`, causing all bodies to fall upward
- **Shockwave** - applies an outward impulse from the canvas centre to every body, demonstrating impulse and momentum
- **Freeze All** - zeroes velocity and angular velocity on all bodies, pausing the simulation mid-frame for discussion
- **Zero Gravity** - removes gravity entirely, useful for orbital mechanics demonstrations
- **Pin Mode** - allows the instructor to click any body to toggle its static state, anchoring it in place as a fixed constraint point

Each event is received by all clients simultaneously, including the instructor, ensuring everyone's canvas stays in sync after the event is applied.

6. Screenshots and Interface

6.1 Login and Registration

The login page uses a glassmorphism card design with gradient accent backgrounds. Users select their role — student or instructor — during registration using a visual card picker. Role selection is locked after account creation and determines the user's permissions and dashboard layout throughout the application.

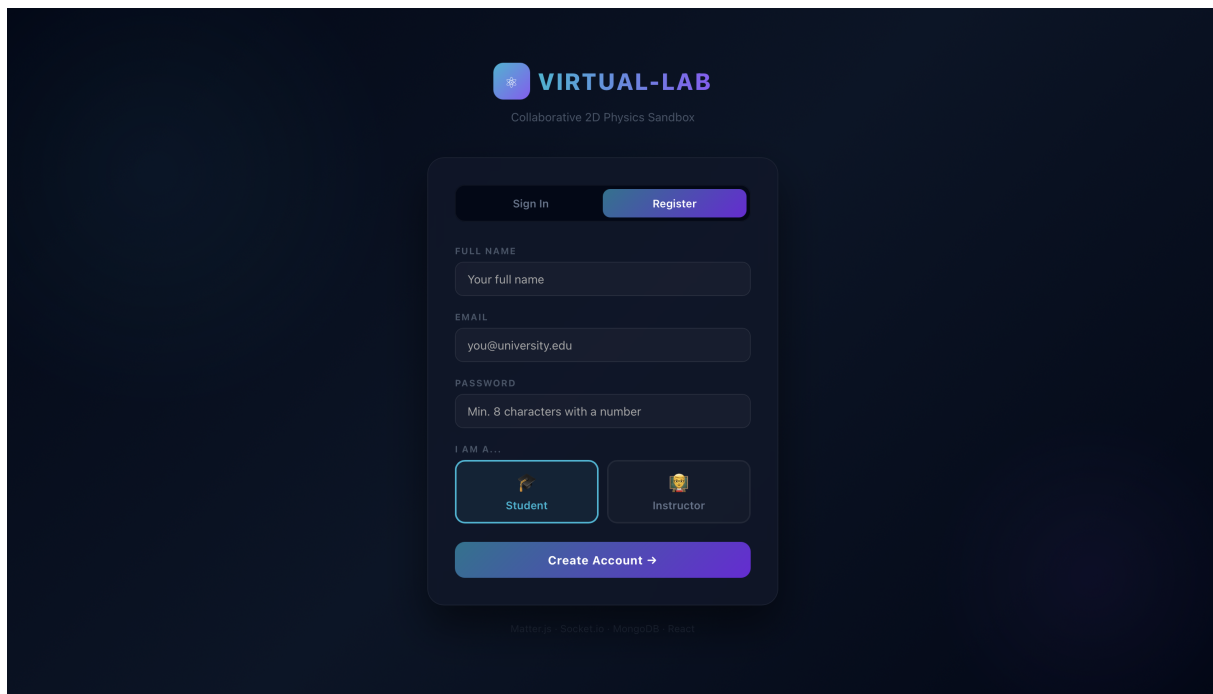


Figure 3: Login and registration page. Role selection (student vs. instructor) determines permissions system-wide. The student dashboard differs from the instructor dashboard in layout and available actions.

6.2 Instructor Dashboard

The instructor dashboard is role-aware. It presents both a Join Room card and a Create Room card side by side, along with a live list of all rooms the instructor has created showing join codes, participant counts, and lock status. The student dashboard shows only the join interface and quick links to the experiment library.

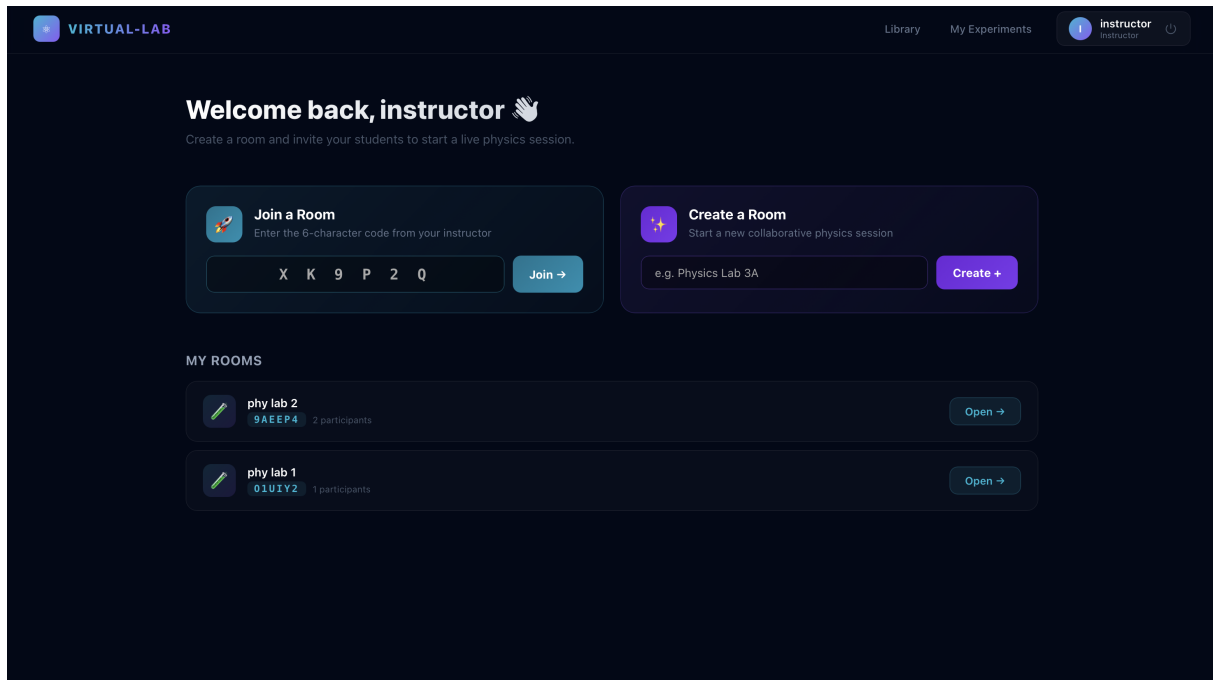


Figure 4: Instructor dashboard showing active rooms with 6-character join codes and participant counts. The student view omits the Create Room card entirely.

6.3 Physics Canvas

The canvas workspace is divided into a left sidebar with physics controls, body tools, and (for instructors) chaos event controls; a central canvas area with live vector overlays; and an energy conservation graph fixed at the bottom. The Controls and Analytics tabs in the sidebar let users switch between parameter sliders and live numerical metrics.

The canvas in Figure 5 shows a pendulum loaded from the Pendulum preset. The green arrow on the bob indicates velocity direction and magnitude; the red arrow indicates acceleration. The energy graph at the bottom is already plotting live data.



Figure 5: Physics canvas with a pendulum loaded. Green arrows show velocity vectors, red arrows show acceleration. The energy conservation graph is visible at the bottom with KE, PE, and total energy plotted live.

6.4 Analytics Panel

The Analytics tab in Figure 6 reveals live numerical metrics for the currently tracked body: velocity, acceleration, position, kinetic energy, potential energy, and total energy, all updated every frame. The graph at the bottom plots all four values simultaneously. The purple dashed total energy line stays flat as the pendulum swings, visually demonstrating that energy is conserved as it transfers between kinetic and potential forms.

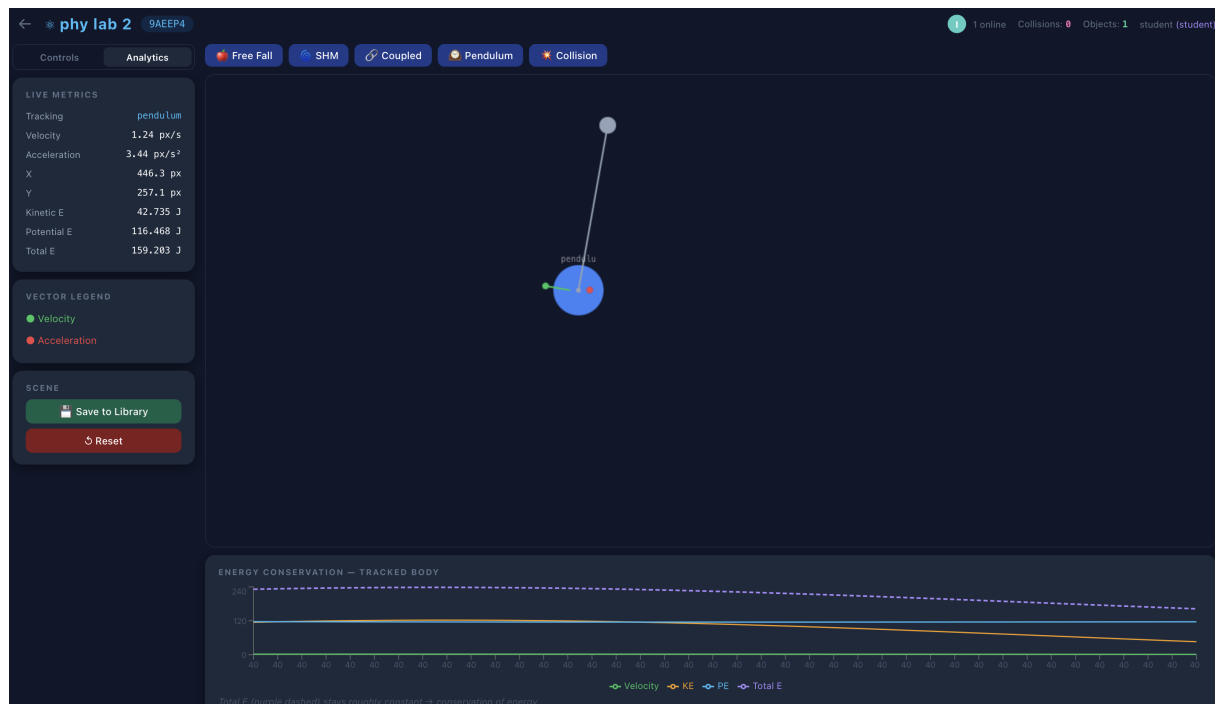


Figure 6: Analytics panel for a swinging pendulum. Kinetic energy (42.7 J) and potential energy (116.5 J) sum to a constant total (159.2 J), visible as the flat purple dashed line on the graph.

7. DevOps and Deployment

7.1 Docker Containerisation

The entire application stack is containerised using Docker. A single `docker-compose.yml` file defines three services - MongoDB, the Node.js backend, and the React frontend - with health checks and dependency ordering so the backend waits for the database to be healthy before accepting connections.

Both Dockerfiles use multi-stage builds. The backend image installs all dependencies in the first stage, then copies only the production subset and source code into a lean final image. The frontend image runs the Create React App build in one stage, then copies the static output into an nginx image for serving - eliminating Node.js from the final container entirely. The nginx configuration includes an SPA fallback rule so that direct navigation to any route (e.g. `/room/XK9P2Q`) serves `index.html` correctly.

A named Docker volume (`virtuallab_mongo_data`) persists MongoDB data across container restarts and rebuilds. This ensures user accounts and saved experiments survive a `docker-compose up -build` cycle.

7.2 CI/CD Pipeline

A GitHub Actions workflow runs on every push to `main` and on every pull request. The pipeline has three sequential jobs:

1. **Backend Tests** - spins up a real MongoDB service container and runs Jest tests with coverage reporting. Integration tests run against a genuine database instance rather than mocks, catching bugs that in-memory stubs would miss.
2. **Frontend Build** - runs `npm run build` to verify the React application compiles without errors.
3. **Docker Build** - builds both Docker images using the GitHub Actions layer cache, confirming the containerisation is valid and that no `COPY` or `RUN` steps are broken.

8. Testing

8.1 Unit Tests - OT Engine

The Operational Transformation engine handles version-based conflict resolution when two clients submit simultaneous operations. It is the most algorithmically complex component in the backend and is tested with nine unit tests covering:

- The fast path (no conflict) - verifying that matching versions are accepted and the server version counter increments correctly
- Conflict resolution when the client is behind the server version - verifying that same-body operations are resolved by timestamp (last write wins)
- Independence of different-body operations - verifying that operations on distinct bodies never interfere with each other
- State mutation safety - verifying that `applyToState` performs a deep clone and does not modify the original state object
- Correctness of each operation type: `ADD_BODY`, `REMOVE_BODY`, `MOVE_BODY`, `SET_GRAVITY`, and `ADD_CONSTRAINT`

8.2 Integration Tests - Auth Routes

Auth route integration tests run the full HTTP request pipeline - middleware, controller, and database - against an in-memory MongoDB instance provided by `mongodb-memory-server`. This means every test runs against real Mongoose schema validation rather than a stub. The suite covers:

- Registration with valid inputs and JWT response
- Duplicate email rejection (409 Conflict)
- Password strength validation - minimum 8 characters, must include a number
- Prevention of admin role self-assignment at registration
- Login with correct and incorrect credentials
- Deliberately vague error messages that do not reveal which credential field failed
- JWT verification on the `/api/auth/me` protected route
- Tampered and expired token rejection

9. Limitations and Future Work

The current implementation is functional and deployable, but several limitations would be addressed before a production release:

Drag conflict resolution. If two users drag the same body simultaneously, the last position update received by the server wins. A proper solution would use operational transformation or vector clocks for continuous position deltas, not just for discrete operations.

Physics determinism. Floating point arithmetic is not guaranteed to be identical across different processors and operating systems. Over a long simulation session, tiny rounding differences can accumulate, causing canvases to diverge slightly. A deterministic physics library or periodic full-state snapshot sync would address this.

Experiment state restoration. While experiments are saved to MongoDB by body position and shape, constraints (springs, pendulum rods) are not reconstructed on reload. A student who saved a spring experiment and returns to it the next day sees bodies but no springs. Full state restoration including constraint relationships is the highest-priority future feature.

JWT storage. Storing the JWT in `localStorage` exposes it to cross-site scripting attacks. A hardened production deployment should use `httpOnly` cookies, which are inaccessible to JavaScript on the page.

Session replay. Recording the sequence of canvas actions with timestamps would allow a complete lab session to be replayed later, which would be valuable both for student review and for instructor assessment.

Mobile support. The physics canvas currently relies on mouse events. Adding touch/-pointer event handling would extend the platform to tablets, which are common in educational settings.

10. Conclusion

VIRTUAL-LAB demonstrates that real-time collaborative physics education is genuinely achievable in the browser with a well-chosen stack and careful attention to synchronisation. The combination of Matter.js for physics, Socket.io for real-time delivery, React for the interface, and MongoDB for persistence produced a system where multiple users can share a live physics experiment not just view a shared screen, but interact with the same simulated world simultaneously.

The engineering challenges encountered were mostly about timing: when the socket connects relative to when React renders, when Matter.js fires events relative to when refs update, when drag events arrive relative to when database state is consistent. Each problem had a clean solution once the timing relationship was understood precisely.

The result is a working EdTech tool that addresses a real gap in online physics education. A class can now load the same pendulum, watch the same vectors, observe the same energy graph, and have the instructor flip gravity on everyone's canvas at once. That is not something existing tools support, and it changes what is possible in a remote or hybrid physics classroom.

Appendix: Technology Stack Summary

Table 2: Complete technology stack

Layer	Technology	Reason for Choice
Frontend	React 18	Component model, Context API for global state
Physics	Matter.js	Best 2D rigid body engine available for browsers
Real-time	Socket.io	WebSocket with automatic fallback, rooms built-in
Charts	Recharts	React-native, handles continuous streaming data cleanly
Backend	Node.js + Express	Non-blocking I/O, ideal for socket-heavy servers
Database	MongoDB + Mon-goose	Flexible JSON schema suits physics state storage
Authentication	JWT + bcrypt	Stateless, horizontally scalable, no session store
Containerisation	Docker	Reproducible builds, one-command full-stack setup
CI/CD	GitHub Actions	Automated test and Docker build pipeline on every PR
Reverse Proxy	nginx	Serves static React build, SPA route fallback